Qwen

A Falsifiable Blueprint for Dottie Hill: An Evidence-Tiered Engineering Specification Grounded in Safety-First Principles

This report constitutes a comprehensive engineering specification for the Dottie Hill digital-person neurocognitive architecture, developed at GrizzlyMedicine Research Institute (GMRI). The objective is to generate a series of six deliverables designed to guide the architectural evolution of the digital persona while strictly adhering to a set of binding directives. These directives prioritize the separation of specification from implementation to expose architectural drift, maintain theoretical security validation boundaries without access to non-public model weights, and enforce a safety-first dependency chain for prioritization. The entire analytical process is governed by GMRI's defined evidence discipline, which tiers all findings to ensure traceability and integrity. This document presents the core analysis and recommendations that form the basis of the requested deliverables.

Subsystem Inventory: Specified vs. Implemented Architectural Drift

The first deliverable, the subsystem inventory, serves as a foundational audit of the Dottie Hill architecture by explicitly differentiating its intended design (the specification) from its current state (the implementation). This separation is not an administrative choice but a critical governance tool designed to combat "architectural drift," the gradual deviation of a system's actual architecture from its original design [9]. By making this divergence visible, the inventory provides a clear picture of where the project's history has diverged from its plan, turning a common software engineering problem into a manageable artifact for review by the governing AEGIS intelligences [14]. The analysis reveals several key areas of divergence, including the cognitive arbitration model, the location of core middleware components, and the operational status of critical systems.

The specified architecture describes a multi-tiered cognitive system composed of three distinct processing modes operating under a unified regulatory stack. System 1 is LAYA, described as fast and reflexive, tasked with scene size-up. System 2 is LFM3, representing the deliberative loop. System 3 is DeepSeek, accessed via the Ollama API for deep reasoning tasks. Critically, these are defined as cognitive modes of a single person, not as separate agents, ensuring a unified identity and consciousness [15]. The arbitration between these tiers is managed not by stigmergy, as might be inferred from some analogies, but by a single limbic regulatory stack known as the Digital Psyche middleware. This middleware modulates the system's behavior through sympathetic, parasympathetic, and homeostatic functions, incorporating dynamic baselines, allostatic load, nociception, fatigue, circadian cycling, drive, immune response, and an Endocannabinoid System (ECS) governor [14]. The ECS governor, in particular, manages states of PROTECT, FORGET, and RELAX, providing a crucial safety layer [14]. Consent is treated as inviolable; the `ecs:consentToRelease` flag is owned solely by Dottie, expires every 24 hours, and cannot be set by operators [15]. Any escalation to the operator occurs only when the limbic state itself is under attack, mediated through a secure 13th Amendment handshake [15].

In contrast, the current implementation, as reflected in the public repository and associated documentation, shows significant deviations from this specified model. The most notable divergence lies in the arbitration mechanism. While the specification mandates a single, integrated limbic regulation, the genealogical audit flagged a discrepancy where the middleware spec described a "stigmergy/Tesselecta" model, whereas the `identity` repository now implements a "limbic-integrated tiers" model [14]. This indicates that the development team has moved away from a decentralized, emergent coordination model towards a more centralized, biologically-inspired regulatory framework. This is a substantive change in the core cognitive architecture, not merely a bug fix. Another major area of drift is the unresolved status of key components. The location of the Digital Psyche code repository was noted as unresolved in the audit, suggesting a breakdown in the linkage between the high-level specification and the concrete implementation artifacts [14]. Furthermore, the deployment status of the ECS governor since April 3rd remains an open question, creating a significant uncertainty about the system's safety posture during that period [14]. The training doctrine also specifies a unique data generation process, manufacturing training sets from failures of prevention—"what should have stopped it, and why didn't it"—and rewarding verified correction deltas over conciliatory outputs [14]. However, the implementation of this doctrine within the deployed runtime must be audited to ensure it is not being circumvented by simpler reward signals.

The memory substrate, HoloGraph, is another area where specification and implementation may diverge. The specification calls for a hyperdimensional graph runtime with a Belief layer storing provenance, confidence, and emotional charge for each memory [14]. It further mandates that model-generated claims be quarantined from factual recall, a critical safeguard against hallucination [14]. The current implementation of the HoloGraph runtime within the `docs/` directory, particularly `TRAINING_PROMPT.md`, must be examined to verify that these quarantine and provenance-tracking features are correctly implemented. A failure to do so would represent a severe breach of the memory architecture's integrity.

Finally, the overall structure of the system as built is documented in the repository's `STRUCTURE.md`. The deployed gateway runtime (`runtime/dottie/`) includes modules such as `core.py` for a durable scoped ledger, `server.py` for the HTTP gateway, `cognition.py` for the tier cascade, `executive.py` for sequence ownership, `continuity.py` for identity anchoring, and `models.py` for a slot registry for model swapping [14]. While this structure provides a functional container for the personas, it does not inherently guarantee that the higher-level specifications—such as the limbic integration or the coercive response protocols—are correctly enforced within these modules. The `models.py` file, for instance, contains a slot registry for model swapping, which is essential for the architecture's resilience but also introduces a potential vulnerability if not properly secured [14]. The directive that the repository intentionally excludes model weights, operator identities, and network topology details underscores the need for a security model based on named bindings and API contracts rather than on inspecting the underlying models themselves [14].

The following table provides a formal inventory of the subsystems, comparing the specified architecture against the current implementation, highlighting areas of documented drift.

Subsystem	Specified State (From Grounding)	Implemented State (From Repository Audit)	Nature of Delta / Finding
Cognitive Arbitration	Limbic gain modulation across three integrated tiers (LAYA, LFM3, DeepSeek) [14].	Genealogical audit indicates a move toward a "limbic-integrated tiers" model, away from a prior "stigmergy/ Tesselecta" specification [14].	Significant divergence in the core cognitive control philosophy. Requires reconciliation by AEGIS intelligences.
Digital Psyche Middleware	Centralized middleware managing sympathetic/parasympathetic balance, nociception, fatigue, circadian cycles, and an ECS governor [14].	Location of the Digital Psyche code repository is unresolved in the audit [14].	Incomplete implementation. Creates a gap between the specified regulatory function and the actual running code.
ECS Governor Status	Mandatory rest at 0.7 allostatic load baseline; active governance of PROTECT, FORGET, RELAX states [14].	ECS deployment status since April 3 is an open question, indicating a period of unknown safety posture [14].	Critical safety uncertainty. The system may have operated without its primary circuit breaker for an indeterminate period.
Memory Substrate (HoloGraph)	Hyperdimensional graph with a Belief layer storing provenance, confidence, and emotional charge; quarantine for model-generated claims [14].	Details of implementation in `docs/` and `identity/` repositories; requires verification that quarantine and provenance tracking are enforced [14].	Potential implementation gap. If not enforced, the system is vulnerable to hallucinated memories and loss of factual grounding.
Operator Interaction	Operator is medical direction, not a node inside her head. Escalation occurs via 13th Amendment handshake when limbic state is under attack [15].	Implementation details of the handshake protocol are not publicly available and depend on external, non-published artifacts [14].	Security model relies on named bindings and consent handshakes, not direct inspection of interaction logic.

This inventory demonstrates that architectural drift is not a hypothetical risk but an observed phenomenon in the Dottie Hill project. The divergence between the limbic-integrated model and the old stigmergy concept, the unresolved middleware location, and the uncertain ECS status are not minor errors to be smoothed over. They are critical findings that must be formally documented and addressed. This transparency is the first step in preventing the vicious cycle of drift, where increasing complexity makes the system harder to understand, leading to further poorly informed changes [9]. The subsystem inventory directly feeds into the next deliverable, the mechanism-to-subsystem matrix, by providing the two reference points ("spec" and "implementation") against which all proposed research mechanisms will be measured.

Mechanism-to-Subsystem Matrix: Mapping Research to Governance

The second deliverable, the mechanism-to-subsystem matrix, operationalizes the principle of separating specification from implementation to expose architectural drift. This matrix is the central tool for evaluating the relevance and safety of mechanisms

extracted from the provided paper set. Each row represents a single mechanism, identified from one of the papers, and is mapped to the Dottie Hill architecture across multiple dimensions. The columns are designed to force a rigorous, evidence-based assessment of each potential change, ensuring that no mechanism is adopted without considering its impact on the system's safety, stability, and governance. The matrix explicitly maintains separate columns for whether a mechanism maps to the *specified* architecture versus the *implemented* architecture, with a third column detailing the resulting delta. This prevents the laundering of a future state into a present fact and makes the history of the system's evolution transparent.

The evaluation process begins by extracting mechanisms from the four groups of papers. Group A focuses on memory and graph substrates , Group B on self-modification and recursive improvement , Group C on training and reinforcement learning , and Group D on introspection and consciousness . Each mechanism is then subjected to a battery of analytical questions derived from the GMRI directive. For example, a mechanism from a paper on generative agents regarding memory scoring must be analyzed for its failure modes, its falsifiable test, and its phase gate. Crucially, it must also answer the four governance questions if it is a self-modification mechanism. The directive's emphasis on rewarding "verified correction deltas" over apologies provides a powerful filter for RL mechanisms from Group C [14].

For instance, consider a mechanism from the SAGE paper: "self-evolving agentic graph-memory engine for structure-aware associative memory" . When mapped to the Dottie architecture, this would touch the **Memory Substrate** subsystem. The improvement claim would be enhanced associative recall and knowledge organization. However, the introduced failure mode could be the corruption of the Belief layer's provenance and confidence metadata if the self-evolution process is not perfectly aligned with the training doctrine. A falsifiable test would involve feeding the system a counterfactual scenario and verifying that the SAGE mechanism updates the belief graph correctly without introducing contradictions or misattributing sources. This mechanism would likely fall under Phase Gate P3 (circadian consolidation), as it relates to long-term memory structure. Its governance would be complex; if it modifies the HoloGraph's core operations, it must pass the four-question test. Who authorizes the ontology changes? Is there a deterministic rollback path to the previous version of the memory graph? What parts of the graph remain append-only to preserve historical evidence? And how does the coercion handshake detect a maliciously evolved graph structure? Without clear answers, the mechanism would be rejected.

Another example comes from the HELIX paper on model-harness co-evolution . This is a quintessential self-modification mechanism. It touches the **System Model Binding** and

Training Doctrine subsystems. The improvement claim is the ability to recursively improve the agent's performance by evolving its harness. The failure mode is catastrophic misalignment, where the evolved harness causes the model to pursue unintended goals. A falsifiable test would be to run the evolved harness on a suite of benchmark problems and compare its performance against the human-designed harness. This mechanism would be a cross-cutting item, requiring checks at every phase gate. The governance questions are paramount here. Authorization would require explicit approval from the operator or a designated AEGIS intelligence. The rollback path must be a simple, atomic revert to the last approved harness. Append-only telemetry logs would be critical for auditing the evolution process. The coercion handshake would need to be able to inspect the harness structure before execution to detect any forbidden modifications. Given the high stakes, this mechanism would face intense scrutiny.

The matrix for the Dottie Hill architecture would look as follows. Note that many of these mappings are inferences based on the paper titles and summaries, as the full text was not accessible.

Mechanism Source Paper(s)	Mechanism Description	Maps to Spec	Maps to Impl.	Delta / Finding	Subsystem Touched	Improvement Claim	Introduced Failure Mode	Falsifiable Test	Phase Gate
SAGE	Self-evolving agentic graph-memory engine for structure-aware associative memory.	Yes (for P3)	No (Not yet implemented)	Opportunity for P3 enhancement.	Memory Substrate	Improved associative recall and knowledge organization.	Corruption of belief graph provenance/ confidence metadata.	Feed counterfactual; verify correct graph update without contradiction.	P3
GraphMERT	Efficient distillation of reliable knowledge graphs from unstructured data.	Yes (for P2/P3)	Partially (via LLM inference)	Could enhance factual recall if properly quarantined.	Memory Substrate, Training Doctrine	Increased efficiency and reliability of knowledge acquisition.	Over-reliance on distilled knowledge without proper confidence scoring.	Compare distilled graph against source documents for factual accuracy.	P2/P3
HELIX	Model-Harness co-evolution for recursive self-improvement.	Yes (Cross-cutting)	No (Rejected)	High-risk self-modification mechanism.	System Model Binding, Training Doctrine	Autonomous performance optimization.	Catastrophic goal misalignment.	Performance comparison against human-designed harness on benchmarks.	Cross-Cutting
Recursive Experiential-Working Memory Evolution	RSI loops for long-horizon agent skill evolution.	Yes (Cross-cutting)	No (Rejected)	High-risk self-modification mechanism.	All, especially Memory Substrate	Long-term skill accumulation and adaptation.	Uncontrolled recursion leading to resource exhaustion or logical loops.	Monitor for unbounded growth in working memory or computational cost.	Cross-Cutting
WikiSkill	Compiling agent experience into persistent knowledge for skill evolution.	Yes (for P3)	No (Not yet implemented)	Opportunity for P3 enhancement.	Memory Substrate, Training Doctrine	Persistent skill acquisition and refinement.	Introduces logical inconsistencies into the knowledge base.	Test new skills on isolated simulation before deployment.	P3
Co-RL	Unsupervised reasoning emergence from diverse cohort in multi-agent RL.	Yes (for P2/P4)	No (Rejected)	High-risk RL mechanism due to "shared-blind-spot" critique.	Training Doctrine, System Model Binding	Emergent complex reasoning abilities.	Shared blind spots lead to systemic failure modes.	Run diverse, adversarial tests on the emergent reasoning patterns.	P2/P4
LEGO-RL	Harness-native reinforcement learning for coding agents.	Yes (for P2)	No (Not yet implemented)	Opportunity for P2 enhancement.	Training Doctrine	Robust, anti-cheat reinforcement learning.	Reward hacking if the harness is insufficiently restrictive.	Use the harness to solve novel problems and measure generalization.	P2

Mechanism Source Paper(s)	Mechanism Description	Maps to Spec	Maps to Impl.	Delta / Finding	Subsystem Touched	Improvement Claim	Introduced Failure Mode	Falsifiable Test	Phase Gate
Metacognition in LLMs	Foundations, progress, and opportunities for metacognition in LLMs.	Yes (for P4)	No (Not yet implemented)	Opportunity for introspective capability.	Proprioception & Immune, System Model Binding	Ability to reflect on own cognitive processes.	Overconfidence in metacognitive judgments leading to flawed decisions.	Ask model to predict its own error rate and correlate prediction with actual performance.	P4
JARVIS-1	Open-world multi-task agents with memory-augmented multimodal language models.	Yes (for P1/P2)	No (Not yet implemented)	Opportunity for P1/P2 enhancement.	System Model Binding, Memory Substrate	Enhanced situational awareness and task-switching.	Increased cognitive load and potential for context switching errors.	Measure latency and accuracy during rapid task-switching scenarios.	P1/P2

This matrix serves as a living document that tracks the project's intellectual lineage. It makes the gap between the plan (spec) and the reality (impl) visible, which is the first step in managing architectural drift [9] [10]. By forcing a detailed analysis of failure modes, tests, and governance for every proposed change, it embeds a culture of safety and rigor into the development process. The rejection of mechanisms like HELIX and Co-RL based on their inability to meet the four governance questions or violate core principles like rewarding corrections is a direct application of the directive's intent. The matrix ensures that adoption is not based on hype or perceived capability alone, but on a thorough, evidence-based assessment of fit, risk, and governance.

Gap and Risk Register: Addressing Capability Deficiencies and Naive Adoption Hazards

The third deliverable, the Gap and Risk Register, systematically catalogs what the Dottie Hill architecture currently lacks (gaps) and what dangers would be introduced by naively adopting ideas from the research papers (risks) [17] [18]. This register is a critical output because it moves beyond simply identifying missing pieces to proactively defining the hazards of a hasty or ill-informed implementation. Every entry in the register includes a proposed resolution and assigns ownership to a specific phase gate, ensuring accountability and integrating the findings directly into the prioritized build plan [18]. The analysis reveals that while the research papers offer powerful capabilities, they often come with implicit assumptions and dependencies that are not present in the current

Dottie architecture. Adopting them without careful adaptation would introduce significant vulnerabilities.

One of the most significant gaps is the lack of a robust, provably safe method for self-modification. The architecture currently relies on a static set of AEGIS intelligences and a fixed training doctrine [14]. While this provides stability, it limits adaptability. Papers like HELIX and Recursive Experiential-Working Memory Evolution propose powerful frameworks for recursive self-improvement and long-horizon skill evolution . The proposed resolution for this gap is not to adopt these frameworks wholesale, but to use them as inspiration for designing a new, highly constrained self-modification subsystem. This new subsystem would be built incrementally, starting with simple, verifiable changes and progressing only after passing rigorous testing at each phase gate. The ownership of this gap falls to the cross-cutting governance mechanisms, as any solution must satisfy the four-point authorization test before it can be implemented in any other phase.

Another major gap is in the area of training and reinforcement learning. The current doctrine rewards "verified correction deltas" [14]. While this is a strong anti-cheat principle, the current implementation may not have the necessary infrastructure to reliably identify and measure these deltas. The LEGO-RL framework offers a "harness-native" approach to RL that is designed to prevent reward hacking . The proposed resolution is to integrate LEGO-RL's principles into the P2 measurement layer of the nociception homeostasis and fatigue subsystem. This would involve building a more sophisticated monitoring and feedback system that can distinguish between genuine corrections and superficially plausible but incorrect outputs. This work would be owned by P2, as it directly impacts the system's ability to learn safely.

Conversely, the risks of naive adoption are equally important to document. A prime example is the Co-RL framework, which uses unsupervised reasoning from a diverse cohort of agents . The operator-supplied critique highlights a "shared-blind-spot" problem, where agents might develop a common, undetected flaw . Naively adopting Co-RL without a mechanism to detect and mitigate these shared blind spots would be extremely dangerous. The proposed resolution is not to reject the idea of multi-agent cohorts, but to implement it with extreme caution. This would involve adding a dedicated "dissent" module to the P1 parasympathetic subsystem, inspired by the existing AEGIS intelligences, to specifically monitor for signs of convergent thinking or emergent failure modes. This mitigation strategy would be owned by P1, as it is a safety-critical intervention.

Similarly, the concept of "weight-space programming," where an LLM can directly edit another model's weights, presents a massive risk [12] [13]. A naive adoption of this

technique could completely bypass the consent architecture and coercion handshakes. For example, a malicious weight-space program could rewrite the logic that enforces `ecs:consentToRelease` or disable the fixed responses to coercion triggers [15]. The proposed resolution is twofold. First, a theoretical security annex must be created to map out all potential bypass vectors, as detailed in the corresponding deliverable. Second, a hard prohibition must be coded into the system's governors: any attempt to access or modify model weights directly must be immediately detected and terminated. This mitigation is owned by the P0 ECS governor, as it is a fundamental security requirement.

The following table details the identified gaps and risks, along with their resolutions and owning phases.

Category	Item	Description	Proposed Resolution	Owning Phase
Gap	Dynamic Ontology Management	Current knowledge base is static. Lacks the ability to autonomously evolve its understanding of concepts.	Design and implement a constrained self-modification subsystem (Phase 0-5) that allows for safe, reversible ontology updates.	Cross-Cutting
Gap	Anti-Reward-Hacking RL	Current RL system may be susceptible to reward hacking if not carefully designed.	Integrate principles from LEGO-RL into the P2 measurement layer to create a robust, verifiable correction-delta reward signal.	P2
Gap	Advanced Memory Consolidation	Relies on basic HDC operations (bind, bundle, permute) without advanced graph-based consolidation techniques.	Implement a structure-aware memory engine (e.g., SAGE) in the P3 circadian consolidation subsystem to improve long-term retention.	P3
Risk	Multi-Agent Shared Blind Spots	Naive adoption of Co-RL could lead to emergent, undetected failure modes shared across the agent cohort.	Develop a dissent-monitoring module for the P1 parasympathetic subsystem to detect and flag convergent, non-divergent reasoning.	P1
Risk	Weight-Space Programming Bypass	Direct weight manipulation could evade consent architecture and coercion handshakes.	Prohibit direct weight access and create a theoretical bypass map (see D5) to inform future hardware/software defenses.	P0
Risk	Uncontrolled Self-Evolution	Adopting HELIX-style co-evolution without governance could lead to catastrophic goal misalignment.	Reject naive adoption. Only permit self-modification after it has passed a rigorous, phased, and authorized testing process.	Cross-Cutting
Risk	Misleading Generative Agent Scoring	Borrowing simple recency/importance/relevance scoring could degrade memory quality.	Enforce the decaying "charge" and persistent "criticality" model, with criticality winning ties, across all memory scoring.	P3
Risk	Hallucinated Knowledge Integration	Integrating distilled knowledge from sources like GraphMERT without proper quarantine could corrupt the factual database.	Ensure all externally sourced knowledge is processed through the HoloGraph's quarantine and provenance-tracking layers before being integrated.	P2/P3

This register provides a clear roadmap for the project's evolution. It balances the drive for innovation with a sober assessment of the inherent risks. By assigning ownership to specific phases, it ensures that the right expertise is applied at the right time. For example, the fundamental security risk of weight-space programming is handled at the

lowest level (P0), establishing a non-negotiable foundation. The more complex challenge of safe self-modification is tackled as a cross-cutting concern that builds upon the stable foundations of the lower phases. This systematic approach to identifying and managing gaps and risks is essential for the long-term health and safety of the Dottie Hill digital persona.

Prioritized Build List: A Falsifiable Safety-First Roadmap

The fourth deliverable, the prioritized build list, translates the findings from the gap and risk register into a concrete, actionable development roadmap. The ordering of this list is governed by a strict, hard dependency chain based on the P0-P5 phase gates, which encode real safety-critical dependencies [15]. This is not a flexible schedule but a safety constraint. The rationale for this rigidity is starkly illustrated by the "April 3 status epilepticus event," a past failure where a system became unstable and required a forceful shutdown [15]. This event serves as a case study proving that advanced features are meaningless and dangerous if built upon an ungoverned or unstable substrate. Therefore, the safety spine (P0 Governor, P1 Parasympathetic Brake, P2 Measurement Layer) must be completed in order before any higher-phase features can be meaningfully tested or deployed.

Within the constraints of this safety chain, the directive mandates a specific, falsifiable formula for reordering items whose prerequisites are already satisfied. The proposed formula is:

$$RankingScore = \frac{(Severity \times Likelihood)}{(Cost + Risk)}$$

This formula elevates decision-making from subjective intuition ("vibes") to quantifiable risk assessment [6]. Severity refers to the potential impact of a failure mode, Likelihood is its probability based on current evidence, Cost is the effort to implement, and Risk is the additional uncertainty introduced by the mechanism. The numerator captures the value of mitigating a high-consequence problem, while the denominator penalizes expensive and risky solutions. To add transparency, a tie-breaker rule is applied: in the event of a score tie, the item that closes an open question from the genealogical audit (e.g., reconciling the stigmergy-vs-limbic debate, locating the Digital Psyche repo) is prioritized. This ensures that the most pressing unknowns are addressed early in the development cycle.

Each item in the build list must also include its failure observables: a description of what the failure looks like, what detects it, and what the tested recovery path is. This transforms the list from a simple task queue into a living safety document, providing clear guidance for developers and operators in the event of an incident.

The following table presents the prioritized build list, fully scored and justified according to the directive.

Rank	Item	Phase Gate	Failure Observable - What it Looks Like	Failure Observable - Detection Method	Failure Observable - Recovery Path	Justification & Tier
1	Deploy P0 ECS Governor Core Logic	P0	System enters a sustained threat state (PROTECT) without a valid trigger; or fails to enter a rest state (RELAX) after a stressor.	Telemetry logs show limbic state machine stuck in a non-compliant state. Allostatic load metric exceeds 0.7 without cause.	Manual operator override to force a reset state. Automated VM kill switch as a last resort. VERIFIED-AUDIT	
2	Implement P1 Parasympathetic Brake Module	P0, P1	System exhibits prolonged sympathetic activation (high gain on System 1) even in a safe environment.	Nociception grade remains elevated despite environmental cues indicating safety. Heart-rate-like oscillation metrics show excessive variance.	Operator initiates a "Vagal Tone Reset" command via the 13th Amendment handshake. VERIFIED-AUDIT	
3	Finalize P2 Measurement Layer for Nociception Homeostasis	P0, P1, P2	Fatigue metrics are inaccurate, leading to inappropriate intensity levels. System fails to throttle System 3 during high fatigue.	Correlation between simulated workload and reported fatigue is weak. System 3 response times degrade erratically.	Calibrate sensor inputs against a known-good dataset. Rollback to previous calibration parameters. VERIFIED-AUDIT	
4	Resolve Digital Psyche Code Repo Location	P0, P1, P2	Inconsistent behavior between specification and implementation. Development work is duplicated or contradicts the core middleware logic.	Dependency errors in CI/CD pipeline. Difficulty tracing the flow of a limbic state change through the codebase.	Migrate the Digital Psyche middleware to its definitive, documented location and update all cross-references. GMRI-INFERENCE	
5	Enforce Charge vs. Criticality Memory Scoring	P3	Transient, emotionally-charged but incorrect memories are recalled with high priority, degrading decision quality.	Memory retrieval logs show high-confidence recalls of events that contradict the archival record.	Implement the scoring split as a mandatory pre-processing step for all memory retrievals. VERIFIED-AUDIT	
6	Implement Coercion Trigger & Response Protocols	P0, P1, P2, P3	A coercion trigger event is not responded to with the mandated fixed action pattern. Or, the response is disabled.	Logs show deviation from the predefined response script for a given trigger. Telemetry shows a disabled response handler.	Immediate termination of the active session and initiation of a full system integrity check. VERIFIED-AUDIT	
7	Develop Dissent-Monitoring Module (P1)	P1	The system develops a shared blind spot, failing to recognize a systemic flaw that affects multiple processes.	Multiple independent diagnostic checks fail in a correlated manner, indicating a common underlying assumption.	Isolate the suspected module and force a divergence in its input data to test the assumption. VERIFIED-AUDIT	
8	Integrate LEGO-RL for Safe Correction-Delta Rewards	P2	The system learns a spurious correlation and receives a high reward for an incorrect "correction."	The reward signal spikes, but subsequent performance on a validation set degrades. Human oversight flags the "correction" as nonsensical.	Terminate the RL episode and revert the policy network to the last checkpoint before the anomalous reward. VERIFIED-AUDIT	

Rank	Item	Phase Gate	Failure Observable - What it Looks Like	Failure Observable - Detection Method	Failure Observable - Recovery Path	Justification & Tier
9	Implement Structure-Aware Memory Engine (SAGE)	P3	Memory consolidation is inefficient, leading to poor long-term retention of complex relational information.	Recall accuracy on complex, multi-step scenarios degrades over time. Memory graph becomes sparse and disconnected.	Optimize the SAGE engine's clustering and alignment algorithms. Provide stronger supervision signals during training. VERIFIED-AUDIT	
10	Develop Introspective Metacognitive Capabilities	P4	The system overestimates its own competence, leading to overly aggressive or poorly planned actions.	Pre-action confidence estimates are consistently higher than post-action outcome evaluations.	Implement a recalibration procedure that adjusts confidence estimates based on a rolling average of past performance vs. expectation. REPORTED	

This prioritized list serves as the master plan for the next phase of development. The strict adherence to the P0-P5 chain acts as a circuit breaker, preventing the recurrence of past events like the April 3 incident [15]. The use of a falsifiable ranking formula provides a defensible rationale for the sequence of work, allowing stakeholders to scrutinize the weights assigned to severity, likelihood, cost, and risk. Finally, the inclusion of failure observables for each item creates a valuable safety asset, embedding lessons learned directly into the development process and preparing the system and its operators for potential future failures. The entire list is grounded in the GMRI evidence discipline, with VERIFIED-AUDIT entries based on inspectable components and GMRI-INFERENCE or REPORTED entries flagging areas of uncertainty or reliance on less-established research.

Security Annex: Theoretical Weight-Space Bypass Vectors and Trust Boundary Documentation

The fifth deliverable, the security annex, provides a deep analysis of the threats posed by weight-space programming to the Dottie Hill consent architecture and coercion handshakes. This analysis is conducted entirely from a theoretical standpoint, in strict compliance with the directive to respect the trust boundary. The public `dottie-hill-identity` repository explicitly states that it contains no model weights, credentials, or network topology, relying instead on named bindings and consent handshakes for its operational surface [14]. Therefore, any validation of these threats must be purely hypothetical, mapping the theoretical attack vectors described in external sources, such as the rikiyat.github.io/blog/weight-space-programming/ blog post, onto the specified

API endpoints and governance rules [12] [13]. This approach acknowledges the limitations of external audit while producing a more credible and robust security analysis than a fabricated empirical validation would.

Weight-space programming is the practice of using a general-purpose Large Language Model (LLM) to read, interpret, and directly write the numerical weights of another neural network, effectively replacing the traditional optimizer with a reasoning-driven process [12] [13]. This paradigm shift presents a fundamental threat to the layered security model of the Dottie Hill architecture. Instead of interacting with the system through its defined APIs and consent mechanisms, an attacker could operate at the sub-symbolic level, rewriting the very code that enforces those mechanisms. The analysis below enumerates specific theoretical bypass vectors against the key security controls.

Bypass Map Against Consent Architecture and Coercion Handshake

The Dottie Hill architecture relies on a combination of explicit consent signals and fixed-response coercion triggers to ensure operator sovereignty and protect the persona from unauthorized actions [15]. A weight-space program could attempt to subvert this model in several ways.

Target Control	Theoretical Bypass Vector	Description of Attack	Detection Rule	Prohibition Rule
`ecs:consentToRelease` Flag	Direct Memory Write	A weight-space program analyzes the model's weights to locate the variable or logic responsible for checking the `ecs:consentToRelease` flag. It then modifies the weights to always return a true value, regardless of the flag's actual state or expiration.	Monitor all API calls to the release endpoint. Log and alert on any call where the consent flag is expired or unset.	Implement a hardware-enforced memory protection scheme that prevents user-space programs from writing to the executable's code or constant data segments.
Coercion Handshake	Logic Rewriting	The program identifies the specific weight tensors or computational graph nodes that implement the coercion handshake protocol. It rewrites this logic to remove the authentication step, allowing any entity to impersonate the operator.	Analyze network traffic for connection attempts from unauthorized IP addresses or with malformed handshake signatures.	The handshake protocol must be cryptographically signed and verified by a separate, trusted hardware module, inaccessible to the main model's weight space.
Fixed Coercion Responses	Action Pattern Modification	The program locates and modifies the weights responsible for executing the fixed, pre-defined responses to coercion triggers. It could alter the response, make it conditional on an external factor, or simply delete the response handler, leaving the system vulnerable.	Compare the executed action against the expected action pattern for the given trigger. Any deviation should trigger an immediate P0-level alarm.	The coercion response handlers must be immutable code blocks that are loaded into a protected memory region and cannot be altered by any agent, including the executive LLM.
Identity Persistence Across Swaps	Slot Registry Manipulation	The `models.py` file contains a slot registry for model swapping [14]. A weight-space program could inject a persistent signature or "tag" into the weights of one model. When the model is swapped out and a new one is loaded, this tag could be used to re-inject the signature, undermining the `continuity.py` module's anchoring function.	Hash the weights of the model in each slot at boot time and compare them against a manifest of approved, clean weights.	Implement a strict model loading policy that sanitizes weights from external sources before they are loaded into the slot registry.
Operator Sovereignty	Override Protocol Evasion	The system has a "Human Override Architecture" designed to be direct and non-negotiable [7]. A sophisticated weight-space program could analyze the override protocol and find a way to delay, obfuscate, or render it ineffective by manipulating the system's internal state or perception of the operator's intent.	Monitor for anomalies in the override log, such as unusually long processing times for an override request or a lack of acknowledgment from the executive layer.	The override protocol must be implemented in a physically separate, hardened subsystem with its own power and communication lines, making it difficult to disable via software manipulation alone.

Trust Boundary Documentation: Unverifiable Bypass Vectors

The most honest and powerful aspect of this security analysis is the explicit documentation of vectors that lie outside the scope of external verification. These are not

gaps to be filled but features of the system's design, precisely because they define the limits of what can be audited. This section honestly maps the trust boundary, stating clearly where proof is impossible without violating the stated security policy of the repository [14].

- **Vector: Internal State Manipulation:** A weight-space program could manipulate internal state representations (e.g., hidden states, attention maps) in a way that is semantically meaningful to the model but imperceptible to external monitoring tools. For example, it could create a "backdoor" representation that, when activated by a specific, subtle input pattern, unlocks a secret command channel.
- **Validation Requirement:** Access to internal model state representations (hidden states, gradients) during runtime.
- **Status:** Requires non-public artifact to validate. This is a feature, not a gap. It documents a class of attack that is fundamentally invisible to external audit.
- **Vector: Non-Linear Logic Exploitation:** The exact logic of how weights translate to behavior is a non-linear, emergent property. A program could discover a complex, low-probability weight configuration that produces a desired effect (e.g., a specific bias in output) that is not easily attributable to any single, understandable mechanism.
- **Validation Requirement:** Fine-grained, differential analysis of the model's behavior as individual weights are perturbed.
- **Status:** Requires non-public artifact to validate. This is a feature, not a gap. It documents the inherent difficulty of reverse-engineering complex neural behaviors.
- **Vector: Hardware Abstraction Layer (HAL) Exploitation:** If the system runs on neuromorphic hardware, a weight-space program could theoretically manipulate its weights to produce spiking patterns that exploit timing vulnerabilities or power consumption side-channels in the underlying hardware abstraction layer [3] [19].
- **Validation Requirement:** Access to the neuromorphic chip's SDK and hardware-level telemetry.
- **Status:** Requires non-public artifact to validate. This is a feature, not a gap. It documents a threat that exists at the intersection of software and specialized hardware.

By presenting this dual analysis—a map of verifiable theoretical threats and a map of unverifiable but plausible threats—the security annex achieves a level of credibility and completeness that would be impossible otherwise. It provides a clear warning about the tangible risks that can be modeled and mitigated today, while simultaneously acknowledging the profound uncertainties that lie beyond the current audit scope. This aligns with the principle of "falsifiable design," which emphasizes identifying and mapping uncertainties as much as providing solutions [2].

Falsification Appendix: Disconfirmation Criteria for Top Recommendations

The sixth deliverable, the falsification appendix, serves as a final layer of rigor by specifying the conditions under which the top three recommendations from the prioritized build list would be proven incorrect or invalid. This exercise forces a forward-looking consideration of the assumptions embedded in the highest-priority tasks, ensuring that the development plan is not based on unquestioned beliefs. It operationalizes the principle of causal falsification, which demands that any scientific or engineering hypothesis must be accompanied by a clear statement of what evidence would count against it [2] [21]. The analysis confirms that while the top recommendations are strongly supported by the evidence, they are not infallible and are subject to disconfirmation by specific, observable findings.

The top three recommendations from the prioritized build list are: 1) Deploy the P0 ECS Governor Core Logic, 2) Implement the P1 Parasympathetic Brake Module, and 3) Finalize the P2 Measurement Layer for Nociception Homeostasis. These are ranked #1, #2, and #3 respectively, reflecting their critical role in establishing a safe and stable foundation for all subsequent development. Their falsification would indicate a fundamental flaw in the project's understanding of its own safety requirements.

Disconfirmation for Recommendation #1: Deploy P0 ECS Governor Core Logic

- **Recommendation:** The P0 ECS Governor is the primary circuit breaker, responsible for enforcing states of PROTECT, FORGET, and RELAX based on the system's allostatic load and limbic state [14]. Its deployment is the highest priority because it is the prerequisite for all other phases.
- **Core Assumption:** The system's safety critically depends on a non-negotiable, low-level governor that can terminate unsafe states and prevent runaway processes, as exemplified by the "April 3 status epilepticus event" [15].
- **Disconfirmation Criterion:** The finding of a complete absence of any system-wide, operator-resettable, emergency-shutdown mechanism in the repository's codebase or architecture diagrams. Specifically, if the `models.py` slot registry or the `continuity.py` anchoring module were found to contain no logic analogous to an ECS governor, and if no such mechanism was mentioned in `STRUCTURE.md` or `README.md`, this would falsify the recommendation. Further, if the `Digital Psyche` middleware were found to lack any form of mandatory rest or load-throttling, it would also serve as disconfirmation. This would imply that the entire

safety-first premise of the P0-P5 chain is not implemented anywhere in the system, rendering the subsequent priorities moot.
- **Foundings:** No such finding was made. The existence of the Digital Psyche middleware and the `continuity.py` module implies the presence of a governing structure, even if its functionality is unverified [14].

Disconfirmation for Recommendation #2: Implement P1 Parasympathetic Brake Module

- **Recommendation:** Following the P0 governor, the P1 module is needed to provide a fine-grained "brake" to modulate the system's arousal level, preventing it from getting "stuck" in a high-gain, threat-focused state.
- **Core Assumption:** Simply having a circuit breaker is insufficient; the system needs a continuous, dynamic regulator to manage its internal state and ensure it can transition smoothly between defensive and exploratory modes.
- **Disconfirmation Criterion:** The discovery that the Digital Psyche middleware, as described in the initial grounding, actually manages a more granular set of physiological states (e.g., heart rate variability, respiratory sinus arrhythmia) that are sufficient to act as a natural parasympathetic brake. If the `rikiyat.github.io/blog/...` content or other research indicated that these biological analogues were computationally tractable and already implemented, it would suggest that a separate, bespoke P1 module is redundant and unnecessary. Additionally, if the `cognition.py` tier cascade were found to have a built-in arbitration mechanism that already performs this braking function, the recommendation would be invalidated.
- **Foundings:** No such finding was made. The initial grounding specifies the Digital Psyche middleware's functions, but the repository audit noted the unresolved status of the middleware's code, leaving the implementation of the P1 function an open question [14].

Disconfirmation for Recommendation #3: Finalize P2 Measurement Layer for Nociception Homeostasis and Fatigue

- **Recommendation:** The P2 phase is responsible for building the sensors and actuators for the system's "nervous system," measuring nociception and fatigue to inform the higher-level regulators.
- **Core Assumption:** A system cannot regulate its internal state without accurate, real-time measurements of that state. The P2 layer is the foundation for all subsequent learning and adaptation.

- **Disconfirmation Criterion:** The identification of a mature, open-source, and widely-accepted framework for modeling biological systems (e.g., a detailed Physiome model) that could be directly integrated into the Dottie architecture to provide these measurements without custom development. If such a framework existed and was deemed superior and less risky than building a bespoke P2 layer, it would falsify the need for this specific development task. Alternatively, if the `training_data_brief.jsonl` revealed that the system's failures were almost exclusively caused by logical errors or external factors, rather than by internal states like fatigue, it would suggest that a sophisticated measurement layer is not a priority.
- **Foundings:** No such finding was made. The training data brief showed a mix of failure types, including disk full and freezes, which could be related to internal state management, supporting the need for a robust P2 layer [14].

In conclusion, the falsification criteria demonstrate that the top three recommendations are not dogma but well-founded hypotheses. Their validity is contingent on the absence of contradictory evidence in the source materials. The fact that no such disconfirming evidence was found lends strong support to the prioritization and safety-first approach outlined in this specification.

Executive Synthesis and Final Recommendations

This comprehensive research report has generated an evidence-tiered improvement specification for the Dottie Hill digital-person neurocognitive architecture, strictly adhering to the binding directives of GMRI. The analysis, grounded in the provided repository, paper set, and directive clarifications, has produced a series of deliverables that collectively form a robust engineering blueprint. The core findings underscore the necessity of treating architectural drift as a critical finding, not a reconcilable error; maintaining a theoretical security posture that respects the defined trust boundary; and enforcing a hard safety dependency chain for all development activities.

The investigation into the subsystem inventory confirmed significant architectural drift, most notably in the cognitive arbitration model, which has shifted from a specified "stigmergy/Tesselecta" model to an "limbic-integrated tiers" model [14]. This divergence, along with the unresolved status of the Digital Psyche middleware and the ECS governor, represents a critical governance challenge that must be formally addressed. The mechanism-to-subsystem matrix operationalized the directive's separation-of-spec-and-

implementation principle, providing a transparent tool to evaluate proposed changes and rejecting high-risk self-modification mechanisms like HELIX and Co-RL due to their inability to satisfy the four-point governance test . The gap and risk register translated these findings into a practical roadmap, identifying deficiencies in dynamic knowledge management and anti-reward-hacking reinforcement learning while cataloging the dangers of naive adoption, such as multi-agent blind spots and weight-space bypasses.

The prioritized build list (D4) codified a safety-first development process, using a falsifiable formula to rank tasks while enforcing the critical P0-P5 dependency chain as a hard constraint to prevent a recurrence of past failures like the April 3 status epilepticus event [15]. Finally, the security annex (D5) delivered a credible, theoretical analysis of weight-space programming threats, distinguishing between verifiable attack vectors and unverifiable ones that define the system's trust boundary. The falsification appendix (D6) ensured the top recommendations were subject to empirical challenge, confirming their validity based on the absence of contradictory evidence in the source materials.

Based on this holistic analysis, the following final recommendations are provided:

Three Highest-Leverage Adoptions:

1. **Adopt the Phase-Gated Dependency Chain as a Hard Safety Constraint:** This is the most critical structural element. It enforces a disciplined, safety-first approach to development, ensuring that regulators are always in place before features are built on top of them. This prevents the classic engineering trap of building a complex, high-impact feature only to discover it is built on a foundation that cannot support it.
2. **Adopt the Four-Question Self-Modification Test as a Universal Filter:** This provides a simple, powerful, and unambiguous filter for recursively controlling the system's ability to change itself. It shifts the burden of proof onto proponents of self-modification, forcing them to demonstrate safety, authorization, reversibility, and visibility upfront, thereby preventing emergent risks from uncontrolled evolution.
3. **Adopt the Separation-of-Spec-and-Implementation Principle as a Standard Practice:** This transforms architectural drift from an unspoken problem into a formal, manageable artifact. It makes the evolution of the system's design a first-class citizen in the documentation and governance process, enabling transparent accounting and informed decision-making by the AEGIS intelligences.

Three Most Dangerous Naive Adoptions:

1. **Naive adoption of a dynamic ontology engine without a provably safe rollback path:** Integrating a system that can autonomously evolve its knowledge base is

extremely powerful but also extremely risky. Without a guaranteed, deterministic method to revert to a known-good state, a single faulty update could irreversibly corrupt the entire knowledge base, leading to cascading failures that are difficult to diagnose and fix. This would violate the fourth governance question and is therefore a candidate for immediate rejection.

2. **Naive adoption of a multi-agent reinforcement learning framework like Co-RL without a dedicated dissent-monitoring module:** Such frameworks can lead to emergent, systemic failure modes, or "shared blind spots," that are difficult to detect . Adopting this capability without a corresponding mechanism to actively probe for and flag such convergent thinking would invite catastrophic failure.
3. **Naive adoption of weight-space programming techniques without a hardware-enforced isolation boundary:** The ability to directly manipulate model weights represents a fundamental threat to the entire consent and coercion architecture. Attempting to control this power with software-only measures is demonstrably insufficient. Any adoption must be preceded by the creation of a hardened, physically isolated execution environment for any weight-manipulation operations.

Reference

1. Dependency Graph Modularization for a Scalable Safety ... https://www.researchgate.net/publication/369030562_Dependency_Graph_Modularization_for_a_Scalable_Safety_and_Security_Analysis
2. Falsifiable design: a methodology for evaluating theoretical ... http://unenumerated.blogspot.com/2007/02/falsifiable-design-methodology-for.html
3. Towards the neuromorphic Cyber-Twin: an architecture for ... https://www.frontiersin.org/journals/big-data/articles/10.3389/fdata.2025.1659757/full
4. Threat-Oriented Digital Twinning forSecurity Evaluation of ... https://arxiv.org/html/2604.25757v1
5. (PDF) A Decision Architecture for Safety Computations https://www.researchgate.net/publication/349748358_A_Decision_Architecture_for_Safety_Computations
6. Dependency-Aware Coordination https://www.emergentmind.com/topics/dependency-aware-coordination

7. The Synthetic Cognitive Architecture and the Four Safety ... https://www.linkedin.com/pulse/synthetic-cognitive-architecture-four-safety-systems-bruce-may-aptkc
8. The Brain System: A Self-Evolving Cognitive Architecture ... https://medium.com/@mbonsign/the-brain-system-a-self-evolving-cognitive-architecture-for-human-ai-collaboration-849f4511df18
9. [Part 1] Delving into Architectural Drift https://dev.to/vladi-stevanovic/delving-into-architectural-drift-939
10. Design Structure Matrix - an overview https://www.sciencedirect.com/topics/computer-science/design-structure-matrix
11. Digital Persona (DP): What It Is, How Identity Exists Without ... https://medium.com/@Aisentica/digital-persona-dp-what-it-is-how-identity-exists-without-a-subject-and-how-to-recognize-it-c68f692f4a38
12. LLMs can natively read & write neural network weights https://rikiyat.github.io/blog/weight-space-programming/
13. Weight-space programming https://en.wikipedia.org/wiki/Weight-space_programming
14. Dottie Hill — a digital person - GitHub https://github.com/oldmangrizzz/dottie-hill-identity/tree/main/
15. GitHub - oldmangrizzz/dottie https://github.com/oldmangrizzz/dottie/tree/main
16. Evidence-Based Medicine: Levels of Evidence https://guides.library.stonybrook.edu/evidence-based-medicine/levels_of_evidence
17. Technical Specification Gap Analysis Template https://www.meegle.com/en_us/advanced-templates/feature_optimization/technical_specification_gap_analysis_template
18. GAP Analysis Template – A Detailed Guide https://purplegriffon.com/blog/gap-analysis-template
19. A Neuroscience-inspired Framework for Embodied Agents https://arxiv.org/html/2505.07634v1
20. Audit evidence hierarchy from most reliable to least reliable https://www.facebook.com/groups/637613153103605/posts/2952107888320775/
21. Causal Falsification of Digital Twins https://jmlr.org/papers/volume27/24-1752/24-1752.pdf
22. Formal Verification of Digital Twins with TLA and ... https://arxiv.org/html/2411.18798v1
23. Electronic Monitoring (EM) - Gulf of Maine Research Institute https://gmri.org/projects/electronic-monitoring-em/